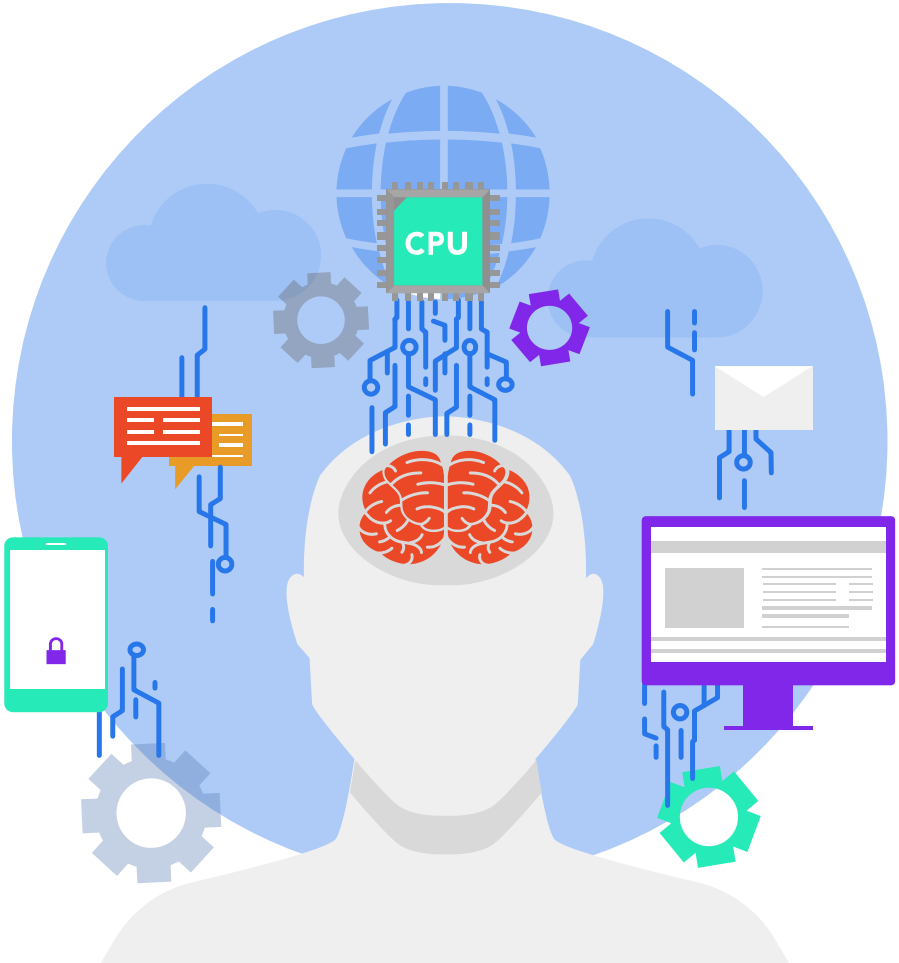



# Nearest-Neighbor Model

**Diyah Utami Kusumaning Putri, S.Kom., M.Sc., M.Cs.**
**Afiahayati, S.Kom., M.Cs., Ph.D.**
Department of Computer Science and Electronics
Universitas Gadjah Mada
diyah.utami.k@ugm.ac.id
afia@ugm.ac.id

# Instance Based Learning



## Instance-Based Learning (IBL)

- **Instance-Based Learning (IBL)** is a machine learning technique that only learns when an input is given for classification.
- IBL is also known as a **lazy learner**, because it does not carry out the learning process (from training data), but directly makes decision (classification result) based on a number of neighbors that has the closest distance (most similar) to the input pattern.
- IBL works locally and can be used for numeric and non-numeric data, discrete and continuous data.
- Examples of instance-based learning algorithms are the ***k-Nearest Neighbor (kNN)***, case-based learning, learning vector quantization, and locally weighted learning.

# Illustration of Instance Based Learning

**Set of Stored Cases (Training Data)**

| Atr1 | Atr2 | .. | AtrN | Class |
|---|---|---|---|---|
| | | | | A |
| | | | | B |
| | | | | B |
| | | | | C |
| | | | | A |
| | | | | C |
| | | | | B |

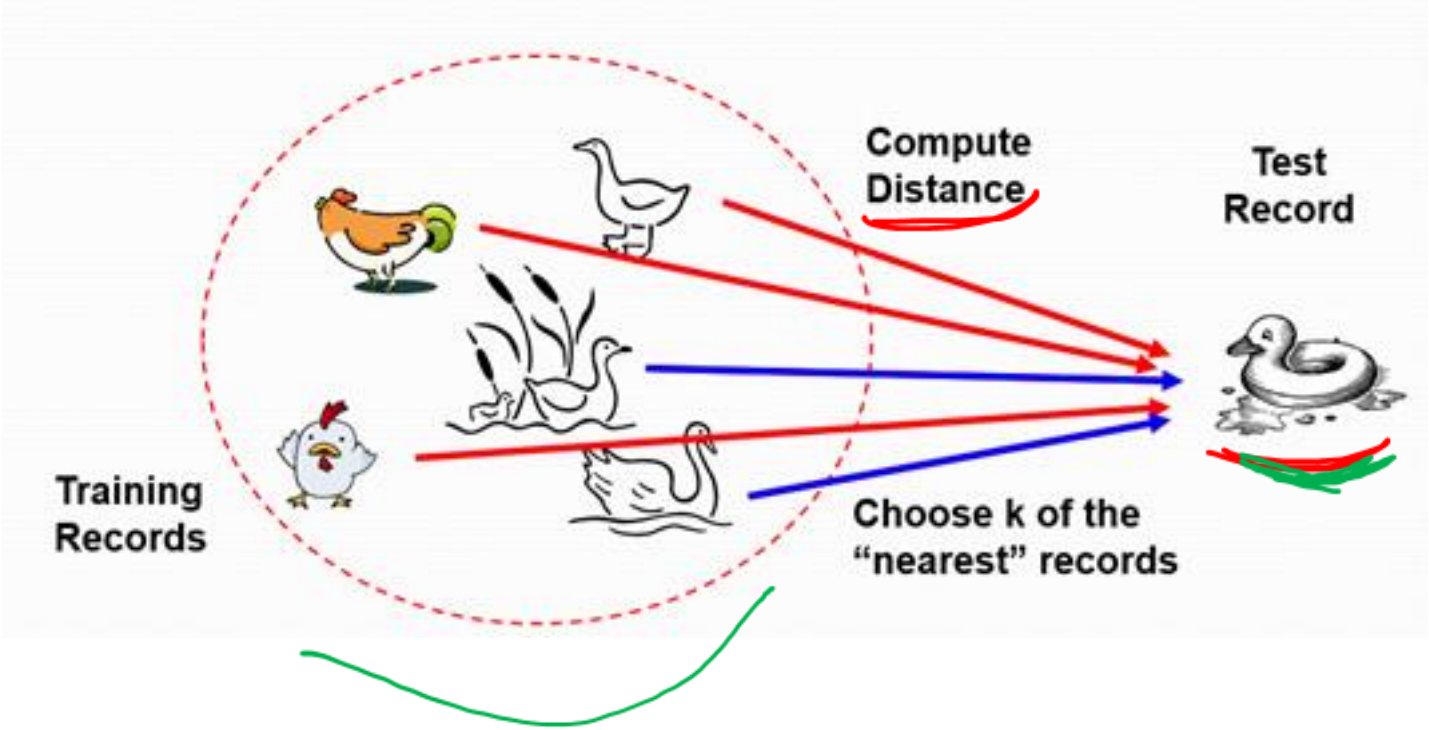
**Unseen Case (Testing Data)**

| Atr1 | Atr2 | .. | AtrN | Class |
|---|---|---|---|---|
| | | | | ?? |

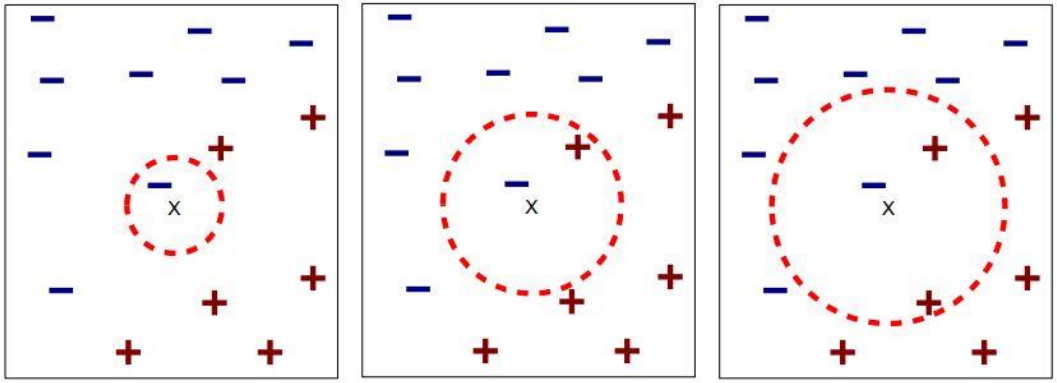
- Store the training data
- Use the training data to predict the class label of the unseen case (testing data)

# Basic Idea of Nearest-Neighbors Model

If it walks like a duck, quacks like a duck, then it's probably a duck.



# Nearest-Neighbor Classifier




(a) 1-nearest neighbor (b) 2-nearest neighbor (c) 3-nearest neighbor

**This algorithm is very simple but not suitable if the data is large. The computational cost is high.**

Training data : 240
Testing data : 60
Many attributes : 10000
Distance calculation = 60 data × 240 data (comparing 10000 attributes for each calculation)

## Requires three things

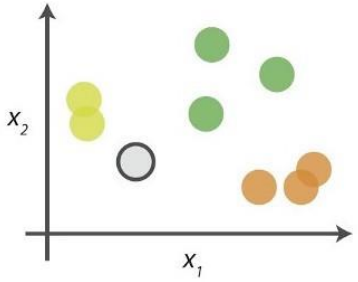
- The set of stored records
- Distance metric to compute distance between records
- The value of *k*, the number of nearest neighbors to retrieve

## Classify an unknown record

- Compute the distance to other training records
- Identify *k* nearest neighbors
- Use class labels of nearest neighbors to determine the class label of unknown record (e.g. by taking majority vote)

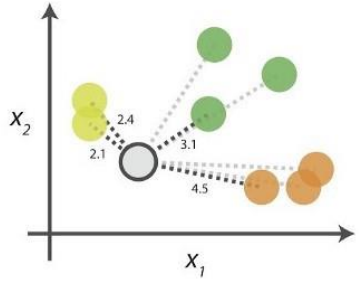
# Nearest-Neighbor Classifier

## 0. Look at the data



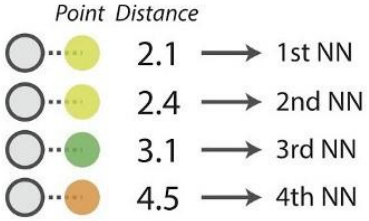

Say you want to classify the grey point into a class. Here, there are three potential classes - lime green, green and orange.

## 1. Calculate distances



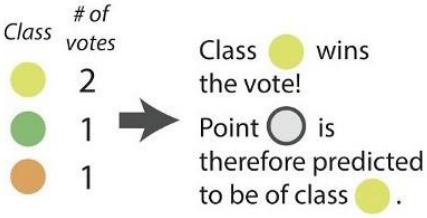

Start by calculating the distances between the grey point and all other points.

## 2. Find neighbours




Next, find the nearest neighbours by ranking points by increasing distance. The nearest neighbours (NNs) of the grey point are the ones closest in dataspace.

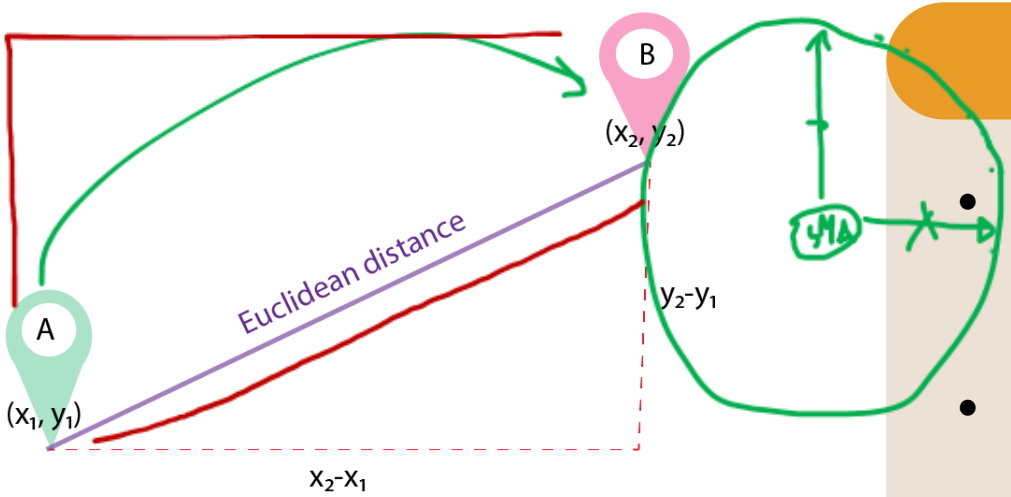
## 3. Vote on labels




Vote on the predicted class labels based on the classes of the k nearest neighbours. Here, the labels were predicted based on the k=3 nearest neighbours.

# Distance Measures

- The **distance measure** is an objective score that summarizes the difference between two objects in a specific domain.
- For the kNN algorithm, to work best on a particular dataset, we need to choose the most appropriate distance metric.
- There are several types of distance measures techniques:
  - Euclidean distance
  - Manhattan distance
  - Hamming distance
  - Minkowski distance (q-norm distance)



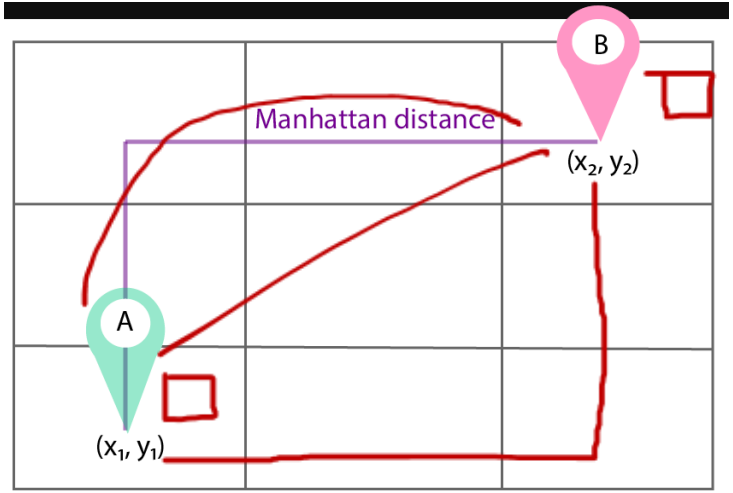

## Euclidean Distance

- **Euclidean distance** is the most widely used one as it is the default metric that *Sklearn library* of Python uses for kNN.
- It is generally used to find the distance between two real-valued (like integer, float, etc...) vectors in Euclidean space.
- One issue with this distance measurement technique is that we have to normalize or standard the data into a balance scale otherwise the outcome will not preferable.

## Equation

$$D = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$D = \sqrt{\sum_{i=1}^{N} (X_i - Y_i)^2}$$



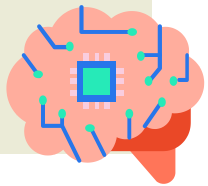

## Equation

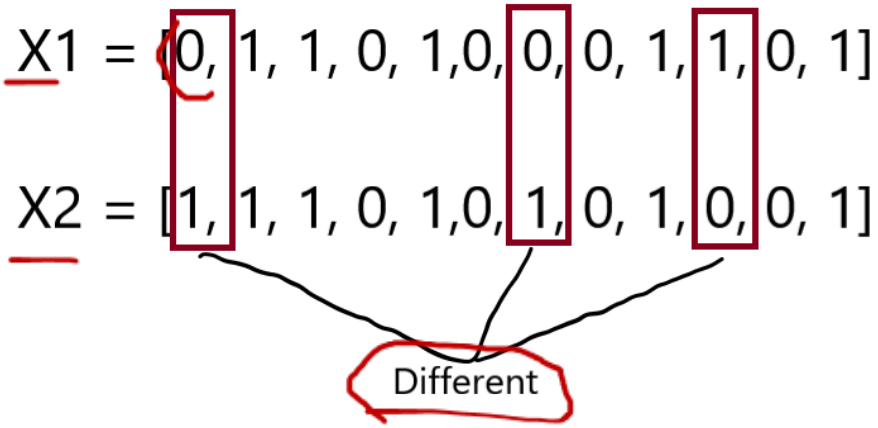
$$D = |x_2 - x_1| + |y_2 - y_1|$$

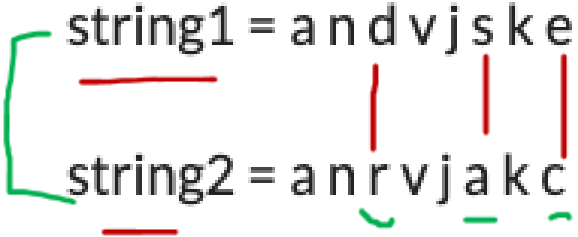
$$D = \sum_{i=1}^{N} |X_i - Y_i|$$

## Manhattan Distance

- This is the simplest technique to calculate the distance between two points, often called Taxicab distance or City Block distance.
- The **Manhattan Distance** calculates the absolute value between two points.
- It calculates the distance exactly as the original path is we didn't take any diagonal or shortest path.

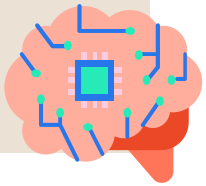





## Hamming Distance

- The **hamming distance** is mostly used in text processing or having the boolean vector (the data is in the form of binary digits 0 and 1).
- This is used mostly when you one-hot encode your data and need to find distances between the two binary vectors, also referred to as binary strings.



## Equation

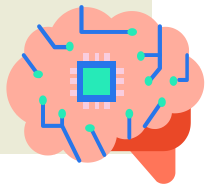
$$D = \left(\sum_{i=1}^{N} |X_i - Y_i|^p\right)^{1/p}$$

The p value in the formula can be manipulated to give us different distances like:

- p = 1, when p is set to 1 we get **Manhattan distance**
- p = 2, when p is set to 2 we get **Euclidean distance**

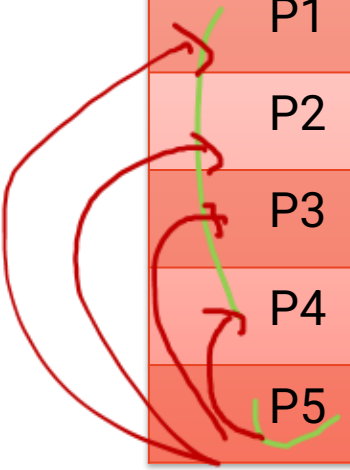
## Minkowski Distance

- The **Minkowski Distance** is a metric intended for real-valued vector spaces.
- We can calculate Minkowski distance only in a normed vector space, which means in a space where distances can be represented as a vector that has a length and the lengths cannot be negative.
- The Minkowski distance is the generalization or generalized form of the Euclidean and Manhattan distance.

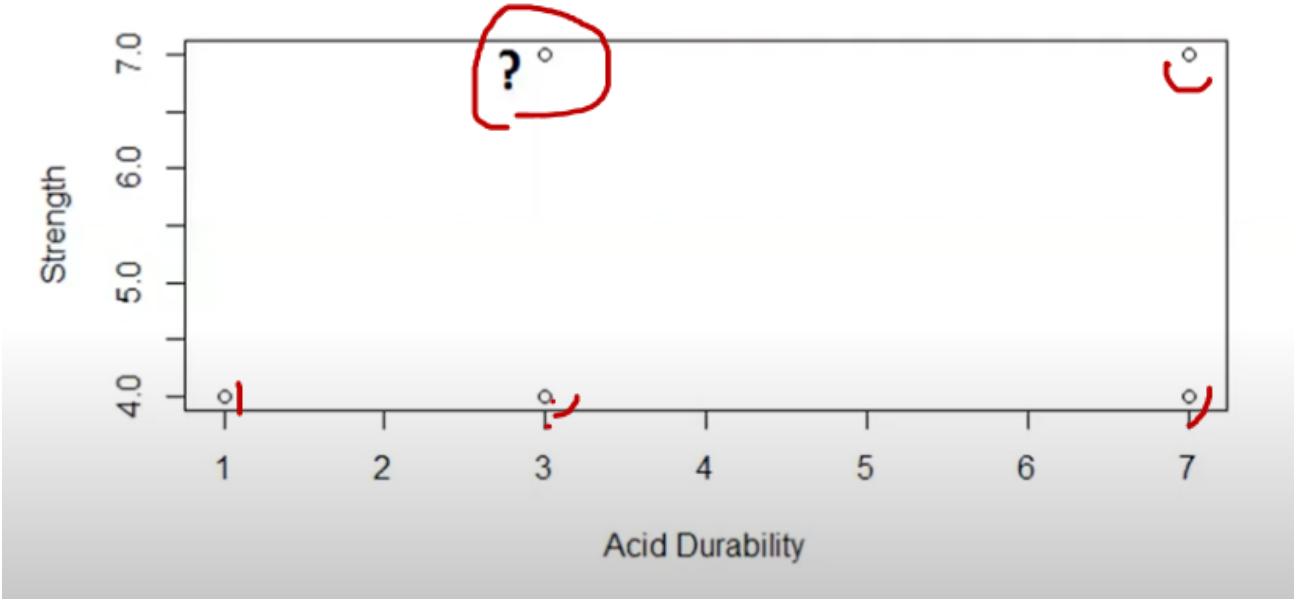


# Example

| Points | X1 (Acid Durability) | X2 (Strength) | Y (Classification) |
|---|---|---|---|
| P1 | 7 | 7 | BAD |
| P2 | 7 | 4 | BAD |
| P3 | 3 | 4 | GOOD |
| P4 | 1 | 4 | GOOD |
| P5 | 3 | 7 | ? |



# Scatter Plot



# Calculate the Distance

## For example, using Euclidean Distance

| Euclidean Distance of P5 (3,7) from | P1 (7,7) | P2 (7,4) | P3 (3,4) | P4 (1,4) |
|---|---|---|---|---|
| | $\sqrt{(7-3)^2+(7-7)^2}$ = $\sqrt{16} = 4$ | $\sqrt{(7-3)^2+(4-7)^2}$ $= \sqrt{25} = 5$ | $\sqrt{(3-3)^2+(4-7)^2}$ $= \sqrt{9} = 3$ | $\sqrt{(1-3)^2+(4-7)^2}$ $= \sqrt{13} = 3.60$ |

# Identify *k* and Determine the Class Label

Identify k – nearest neighbors. Then, use class labels of nearest neighbors to determine the class label of unknown record.

par. tuning

## For example, 3-nearest neighbors

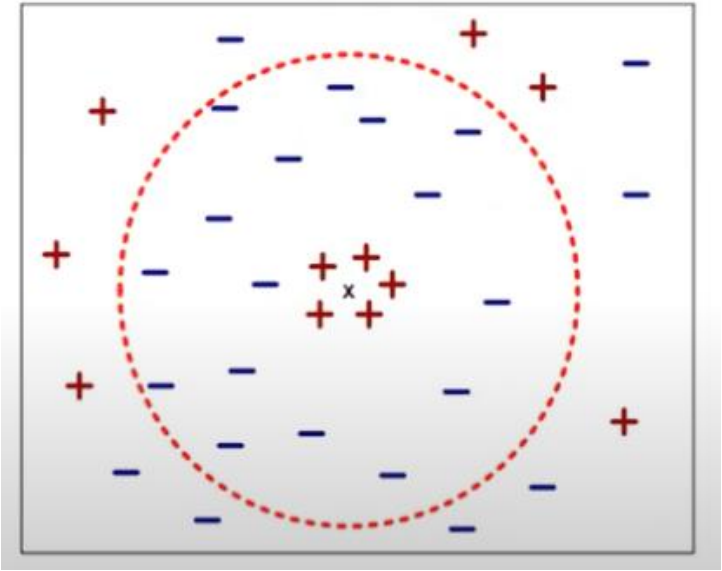
| | P1 (7,7) | P2 (7,4) | P3 (3,4) | P4 (1,4) |
|---|---|---|---|---|
| Euclidean Distance of P5 (3,7) from | $\sqrt{(7-3)^2+(7-7)^2}$ $= \sqrt{16} = 4$ | $\sqrt{(7-3)^2+(4-7)^2}$ $= \sqrt{25} = 5$ | $\sqrt{(3-3)^2+(4-7)^2}$ $= \sqrt{9} = 3$ | $\sqrt{(1-3)^2+(4-7)^2}$ $= \sqrt{13} = 3.60$ |
| Class | BAD | BAD | GOOD | GOOD |

# Result

| Points | X1 (Acid Durability) | X2 (Strength) | Y (Classification) |
|---|---|---|---|
| P1 | 7 | 7 | BAD |
| P2 | 7 | 4 | BAD |
| P3 | 3 | 4 | GOOD |
| P4 | 1 | 4 | GOOD |
| P5 | 3 | 7 | GOOD |

# Issue: Choosing K

- If k is too small, sensitive to noise points.
- If k is too large, neigborhood may include points from other classes.



# Issue: Lazy Learner



The k-NN classifier is a lazy learner

- It does not build models explicitly, unlike eager learners such as decision tree induction and rule-based systems.
- Classifying unknown records is relatively expensive, especially if the data and its features are large because we need to calculate distances for each point.

# The Drawbacks of kNN

- Sensitive to less relevant features
- Sensitive to neighbor size *k*
- Sensitive to noise data and outlier data
- Relatively high computational and time complexity to find the nearest neighbor among all training data every time doing classification
- Relatively large memory complexity to store all training data

# Improving kNN Performance

- Improvements with the distance measure, to reduce sensitivity to less relevant features.
- Improvements with neighboring size to reduce the sensitivity of kNN to neighboring size *k*.
- Improvements with class probability estimation, to reduce the sensitivity of kNN to noise data and outlier data.
- Improvements with data structure, to reduce time and memory complexity.